

SecureDoc Project - Step by Step Explanation

STEP 1: Create Project Folder

You make a folder like:

- Why?
- Keeps your project organized
- Easy to run everything from one place

```
SecureDoc/  
■■■ app.py  
■■■ requirements.txt
```

STEP 2: requirements.txt

This file contains all libraries your project needs:

- streamlit → UI (frontend)
- langchain → connects everything (brain)
- openai → AI model (LLM)
- chromadb → stores embeddings
- pypdf → reads PDF
- tiktoken → token processing
- Why? Instead of installing one by one, you just run:

```
pip install -r requirements.txt
```

STEP 3: API Key

This is the most important step. You add:

- Why?
- AI model (LLM) needs internet + API
- Without key → project will NOT work

```
os.environ["OPENAI_API_KEY"] = "your_key"
```

STEP 4: Upload PDF

- What happens?
- User selects PDF from system
- File is uploaded into your app

```
uploaded_file = st.file_uploader("Upload your PDF")
```

STEP 5: Save PDF Temporarily

- Why?
- System needs file path to read PDF
- So we save it locally first

```
with open("temp.pdf", "wb") as f:  
    f.write(uploaded_file.read())
```

STEP 6: Load PDF

- What happens?
- PDF → converted into readable text
- Stored as "documents"

```
loader = PyPDFLoader("temp.pdf")  
documents = loader.load()
```

STEP 7: Text Chunking

- Why needed?
- LLM can't handle very large text at once
- Break text into small pieces (chunks)
- Keep overlap → so context is not lost

```
splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=100)  
texts = splitter.split_documents(documents)
```

STEP 8: Create Embeddings

- What is embedding?
- Text → converted into numbers (vectors)
- Why?
- Machine understands numbers better
- Helps in searching similar content

```
embeddings = OpenAIEmbeddings()
```

STEP 9: Store in Vector DB

- What happens?
- All chunks stored in ChromaDB
- Each chunk has its vector

- Why? Enables fast semantic search

```
db = Chroma.from_documents(texts, embeddings)
```

STEP 10: Create QA System

- What happens?
- Connects Database (knowledge) + LLM (brain)
- Core of your project

```
qa = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(),
    retriever=db.as_retriever()
)
```

STEP 11: User Question

- What happens?
- User types question
- System prepares to answer

```
query = st.text_input("Ask question")
```

STEP 12: Retrieval + Answer

- Behind the scenes:
- Question → converted to embedding
- Similar chunks found from DB
- Chunks sent to LLM
- LLM generates final answer

```
answer = qa.run(query)
```

COMPLETE FLOW

First, the PDF is converted into text and split into smaller chunks. These chunks are converted into embeddings and stored in a vector database. When a user asks a question, the system retrieves relevant chunks and sends them to the LLM, which generates a context-based answer.

KEY POINTS

- Not simple chatbot
- Uses RAG (Retrieval-Augmented Generation)
- Answers come from your document only